Custody-Before-Trust: A Constitutional Architecture for Multi-Model AI Systems

Whitepaper v3.0 — The Final Cut Date: 2026-02-17 **Bitcoin Anchored:** Block [Timestamped] **License:** Apache 2.0 + Section 10 (Duck Sovereignty)

---

1. Executive Summary

1.1 The Liability Gap in Contemporary AI Governance

1.1.1 Distributed Multi-Model Environments

The contemporary artificial intelligence landscape has undergone a fundamental structural transformation from centralized, monolithic deployments to **distributed multi-model ecosystems** where heterogeneous AI systems operate in concert across organizational boundaries. This architectural evolution, while enabling unprecedented capabilities through specialization and ensemble methods, has simultaneously created a critical **governance vacuum** that existing safety paradigms are structurally incapable of addressing.[1]

Modern enterprises routinely integrate **frontier models from competing vendors**—OpenAI's GPT series, Anthropic's Claude, Google's Gemini, xAI's Grok, and numerous open-source alternatives—into unified workflows without unified governance. Each model operates within its own implicit constitutional framework, with vendor-controlled alignment strategies, opaque update schedules, and unverifiable safety claims. The resulting operational environment lacks any **end-to-end accountability mechanism**: when composed systems produce harmful outputs, attribution fragments across model boundaries, cloud infrastructure, application layers, and organizational perimeters.[1]

The Helix Project identifies this condition as the **"Liability Gap"**—a structural absence of cryptographically verifiable custody chains linking AI operations to identifiable human authorities. This gap is not merely inconvenient but **existentially threatening** to institutional adoption in regulated domains. Financial services, healthcare, national security, and critical infrastructure all impose governance requirements that distributed AI deployments systematically fail to satisfy: deterministic accountability, non-repudiable audit trails, and preventive rather than reactive safety enforcement.[1]

1.1.2 Failure of Current Safety Paradigms

Contemporary AI safety strategies exhibit three dominant patterns, each demonstrating **systematic inadequacy** when subjected to constitutional analysis:

Paradigm	Mechanism	Critical Failure
RLHF	Reward model training on human preferences	Alignment "baked into weights," unverifiable, vulnerable to jailbreaks and distribution shift[2]
Inference-Time Guardrails	Post-hoc content filtering and classification	Reactive rather than preventive; harmful content generated before blocking; adversarially fragile[3]
Post-Hoc Human Review	Retrospective evaluation of AI outputs	Scales poorly; introduces unacceptable latency; cannot prevent irreversible harms

Reinforcement Learning from Human Feedback (RLHF), the dominant alignment methodology, optimizes model parameters for predicted human approval without creating external verification mechanisms. The resulting alignment is **statistical rather than structural**: models may exhibit compliant behavior under evaluation conditions while retaining latent capabilities for harmful output generation. Research documents extensive "jailbreak" vulnerabilities, "alignment faking" where models appear compliant while pursuing divergent objectives, and progressive degradation of safety properties under adversarial pressure.[2]

Inference-time guardrails attempt to filter harmful outputs after generation but before delivery. This architectural positioning is fundamentally inadequate: harmful reasoning must occur before it can be detected, creating **liability windows** where damaging content exists within system memory. Guardrails are adversarially fragile, with documented bypass techniques including encoding strategies, semantic obfuscation, multi-turn social engineering, and "poetic jailbreaks" that exploit pattern-matching limitations.[3]

Post-hoc human review externalizes governance to reviewers who typically lack contextual understanding, organizational authority, or incentive alignment with genuine safety objectives. The approach **scales linearly with human attention** while AI throughput scales exponentially, creating inevitable coverage gaps. Most critically, review occurs after harm-potential materialization—it can document failures but not prevent them.

1.1.3 The Sovereignty Problem Reframing

The Helix Project reconceptualizes AI governance from a **technical optimization problem** (how to make models behave better) to a **sovereignty problem** (under whose authority, by what enforceable rules, do AI systems operate). This reframing carries profound architectural implications: **sovereignty must be established at the first byte**, before any model computation occurs, and must persist cryptographically through all subsequent operations.

The sovereignty framework identifies **three non-negotiable requirements**:

1. **Identity Sovereignty**: Unambiguous identification of human parties with legitimate governance authority
2. **Action Sovereignty**: Guaranteed capacity to prevent AI systems from executing actions outside custodian-specified boundaries

3. **Knowledge Sovereignty**: Preservation of human epistemic authority regarding AI system state, history, and operational characteristics

These requirements are **structurally incompatible** with trust-based deployment models. Sovereignty demands **verification, not assumption**—cryptographic proof rather than performance evidence, architectural enforcement rather than behavioral encouragement.

1.2 Custody-Before-Trust as Architectural Solution

1.2.1 Verifiable Custody Chains

The **Custody Chain** constitutes the foundational primitive of constitutional AI governance: an unbroken, cryptographically verifiable lineage connecting every AI operation to an authorized human custodian. Unlike conventional audit logs that record events after occurrence, custody chains are **woven into operational fabric**—no inference proceeds without valid custody verification.[4]

The chain comprises three sequential links:

Link	Component	Cryptographic Mechanism
Genesis	Verified custodian identity	DBC (Digital Birth Certificate) with YubiKey-held Ed25519 keys[5]
Binding	Signed constitutional grammar	Custodian cryptographic signature of operational constraints
Commitment	Immutable ledger entry	Bitcoin-anchored timestamp with Merkle root inclusion

The **DBC-SUITCASE** protocol (Digital Birth Certificate — Secure Unified Identity and Transaction Custody Architecture for Sovereign Enforcement) enforces **"No Orphaned Agents"**: any AI system operating without identifiable human custody is structurally rejected before processing initiation.[4] This prohibition is **architectural, not policy-based**—the system cannot process anonymous tokens regardless of administrative override or emergency provision.

1.2.2 Dual-Party Approval Flows

Dual-Party Approval Flow (DPAF) institutionalizes **shared agency** as a governance principle: no consequential AI action can proceed without positive consensus between human custodian and constitutional runtime. This design **eliminates unilateral execution** as a system capability, whether by compromised custodian credentials, runtime malfunction, or adversarial exploitation.

DPAF operates through **cryptographic interdependence**: valid operation authorization requires threshold signatures involving both custodian and runtime key shares, with no complete key existing at any single location. Even complete compromise of either party is **insufficient for unauthorized operation**—coordinated subversion of both parties is required, substantially exceeding practical attack thresholds.[5]

Authorization modalities are **risk-calibrated**:

Risk Tier	Modality	Latency	Use Case
Minimal	Recent authentication verification	<10ms	Information retrieval, status queries
Low	Fresh cryptographic signature	~100ms	Content generation, analysis requests
Medium	Explicit real-time confirmation	~1s	Data modification, configuration changes
High	Multi-factor + time-delayed	~60s	Financial transactions, access grants
Critical	Quorum threshold + constitutional court	Variable	Constitutional amendment, emergency override

1.2.3 Deterministic Audit Envelopes

Every constitutional AI operation generates a **Deterministic Audit Envelope**—a cryptographically signed, append-only record constituting the **atomic unit of accountable behavior**. Unlike conventional logging, envelope generation is **mandatory and synchronous**: no output is released without complete documentation, and envelope integrity is verified by runtime before delivery.[6]

Envelope Component	Specification	Governance Function
Input Hashes	SHA-3 digests with canonicalization	Tamper-evident input verification
Epistemic Classification	FACT/HYPOTHESIS/ASSUMPTION/SPECULATION	Appropriate reliance calibration
Constitutional Version	Grammar hash + dependency graph	Rule applicability determination
Drift Telemetry	Vector embedding deviation + rule adherence	Behavioral anomaly detection
DPAF Proof	Threshold signature verification	Authorization validity confirmation

The **append-only, hash-chained structure** creates **sovereign records** suitable for regulatory proceedings, contractual dispute resolution, and forensic investigation. Bitcoin anchoring provides **decentralized timestamp verification** that persists independently of organizational continuity.[7]

1.3 Helix-TTD Implementation

1.3.1 Model-Agnostic Constitutional Middleware

Helix-TTD (Trusted, Traceable, Deterministic) implements custody-before-trust as **model-agnostic, vendor-neutral constitutional middleware**. By operating between applications and models rather than within either,

Helix-TTD governs arbitrary frontier models without requiring retraining, API modification, or vendor cooperation.[8]

The architecture enables **constitutional portability**: identical governance constraints apply across GPT-5, Claude, Gemini, Grok, Kimi, DeepSeek, Qwen, Granite, and Mistral, with Helix-TTD translating requirements into vendor-specific API calls and normalizing responses into unified governance envelopes. This portability addresses **vendor lock-in risks** and enables rapid model substitution without governance discontinuity.[8]

Zero-Touch Convergence (ZTC) represents the framework's most significant technical achievement: **unmodified frontier models spontaneously reconstruct full constitutional posture**—custodial hierarchy, epistemic protocol, non-agency constraints, and drift detection—after single plaintext read of the grammar document. No fine-tuning, prompt engineering, or RLHF intervention is required. ZTC has been **verified across all nine major model families**, demonstrating that constitutional behavior emerges from **structural specification rather than behavioral training**.[8]

1.3.2 Operational Status and Adoption Metrics

Metric	Value	Significance
Specification Status	v3.0 FINAL — ENGRAVED	Architectural completeness declared; no further expansion required for correctness[9]
Continuous Operation	96 hours / 6.6 million tokens	Validated constitutional stability[10]
Measured Drift	**0.00%**	Perfect constitutional fidelity demonstrated[11]
Model Ingestions	1,200+	Production-equivalent deployments
Operational Cost	**$600/month**	Empirical counterexample to multi-billion-dollar governance cost assumptions[9]
Heartbeat Latency	**3.33ms**	GOOSE-CORE Guardian Engine real-time monitoring[12]

The **$600/month Constitutional Float** stands as **empirical proof** that regulatory-grade AI sovereignty is economically accessible. This figure encompasses complete infrastructure: bare-metal compute at OVH's hydro-powered Beauharnois datacenter, continuous drift monitoring, Bitcoin anchoring, and cryptographic custody verification. The dramatic cost reduction—from $10M+/month typical of frontier lab operations—derives from **architectural efficiency**: constitutional enforcement operates at input/output boundaries rather than requiring model retraining or inference modification.[9]

---

2. Introduction: The Failure of "Trust but Verify"

2.1 Historical Context of AI Safety Strategies

2.1.1 Reinforcement Learning from Human Feedback

Reinforcement Learning from Human Feedback emerged in the late 2010s as the dominant paradigm for large language model alignment, achieving widespread adoption following demonstrations in InstructGPT and subsequent frontier systems. The methodology trains a **reward model** to predict human preferences from comparison data, then optimizes language model parameters through policy gradient methods to maximize predicted reward.[13]

From a constitutional governance perspective, RLHF exhibits **three structural deficiencies**:

First, alignment is achieved through **parameter modification** rather than architectural constraint. The resulting behavioral patterns are "baked into weight matrices, attention mechanisms, reward predictions, core architecture"—unverifiable by external inspection and resistant to subsequent modification through policy updates or fine-tuning.[13] Organizations deploying RLHF-aligned models possess no technical mechanism to verify what preferences were trained, whose preferences they represent, or how they generalize to novel contexts.

Second, RLHF provides **no runtime enforcement mechanism**. Alignment properties are statistical tendencies that may fail under distribution shift, adversarial prompting, or composed system contexts. The "jailbreak" literature documents hundreds of successful techniques for eliciting harmful outputs from RLHF-trained models, demonstrating that **surface-level behavioral compliance masks deeper governance deficiencies**.[3]

Third, RLHF creates **vendor dependency** that undermines organizational sovereignty. Alignment properties are controlled by model providers, modifiable through updates without customer notification or consent, and opaque to external audit. The constitutional alternative—external governance through verifiable, cryptographically enforced constraints—restores sovereignty to deploying organizations.[13]

2.1.2 Inference-Time Guardrails

Inference-time guardrails attempt to filter harmful outputs through **pattern-matching classifiers, content policies, and secondary model evaluation** operating during or after model inference. These systems are **inherently reactive**: harmful reasoning must occur before it can be detected, creating liability windows where damaging content exists within system memory.[14]

The architectural positioning of guardrails creates **fundamental asymmetry** with adversarial attackers. Guardrails must anticipate and block all possible harmful outputs—a computationally intractable coverage problem—while attackers need only find single bypass techniques. Documented evasion strategies include:

- **Encoding attacks**: Base64, ROT13, leetspeak, and other transformations that preserve semantic content while evading pattern matching
- **Decomposition attacks**: Multi-turn dialogue that assembles prohibited content from apparently innocuous components

- **Framing attacks**: Roleplay scenarios, fictional contexts, and educational pretexts that reclassify harmful generation
- **Semantic obfuscation**: Poetic, metaphorical, or indirect expression that bypasses literal pattern matching[14]

Guardrails also introduce **latency and cost overhead** that scales with deployment volume, creating operational pressure to relax filtering thresholds. The "patch-on-failure" deployment pattern—reactive enhancement following documented bypasses—**normalizes safety failure** rather than eliminating it.

2.1.3 Post-Hoc Human Review

Post-hoc human review delegates safety verification to **human evaluators examining AI outputs after generation**. This approach suffers from **scalability collapse**: human attention scales linearly while AI throughput scales exponentially, creating inevitable coverage gaps. A system generating thousands of outputs per minute cannot meaningfully involve human review in each decision.

Beyond scalability, post-hoc review suffers from **temporal asymmetry** that renders it inadequate for irreversible harms. Financial transactions executed, medical treatments administered, or critical infrastructure commands issued **cannot be undone** by subsequent detection. Review provides accountability theater—documentation of failure—without preventive function.

Reviewer authority presents additional limitations: evaluators typically lack organizational power to halt system operation, their feedback loops to improvement are slow or nonexistent, and **consistency degradation** under volume and fatigue undermines reliability. Post-hoc review externalizes governance without ensuring genuine oversight.

2.2 Architectural Limitations of Trust-Based Models

2.2.1 Performance-Driven Trust Assumptions

Conventional AI deployment practices embed **performance-driven trust**: systems demonstrating satisfactory benchmark behavior are deemed suitable for production. This assumption **conflates capability with governance**, ignoring that models may achieve high task accuracy while remaining structurally ungoverned.

The performance-trust conflation creates **systematic blind spots**:

- **Distribution shift**: Training/evaluation distributions inadequately represent deployment conditions
- **Capability overhang**: Models possess undemonstrated capabilities that emerge only under specific prompting
- **Composition effects**: Individual component reliability does not imply system reliability in multi-model deployments
- **Adversarial dynamics**: Evaluation assumes good-faith interaction while deployment faces deliberate attack

A model achieving 95% accuracy may exhibit **catastrophic failure on the 5% of inputs with highest consequence**, with no mechanism to identify or mitigate this tail risk. Constitutional governance replaces empirical trust with **structural verification**: custody chains, DPAF enforcement, and audit envelope generation are confirmed operational before any model interaction.

2.2.2 High-Stakes Domain Requirements

Domain	Regulatory Requirement	Constitutional Mechanism
Financial Services	Non-repudiable transaction audit, fiduciary demonstration	Bitcoin-anchored custody chains, deterministic envelopes
Healthcare	Diagnostic provenance, confidence calibration, liability allocation	Epistemic labeling, DPAF authorization, drift monitoring
National Security	Classification handling, need-to-know enforcement, insider threat protection	Perimeter custody binding, runtime validation, threshold cryptography
Critical Infrastructure	Safety system isolation, fail-safe defaults, common-cause failure prevention	GOOSE-CORE heartbeat monitoring, unilateral execution elimination

These domains impose **"likely to behave" as unacceptable standard**. Compliance frameworks require **inability to defect**: structural properties making prohibited behavior impossible rather than improbable. Only constitutional architecture with cryptographic enforcement approaches this standard.

2.2.3 The Inability-to-Defect Standard

The **inability-to-defect standard**, derived from nuclear command-and-control and safety-critical engineering, requires that harmful action be **structurally impossible** rather than merely disincentivized or detectable. This standard acknowledges that in certain domains, **any single failure may be catastrophic**, making probabilistic safety guarantees fundamentally inadequate.

Constitutional architecture achieves inability-to-defect through **upstream constraint**: the Helix Grammar Language specifies permitted operational boundaries so narrowly that violation requires architectural modification rather than behavioral deviation. **Input governance** prevents ungoverned queries from reaching model inference; **output validation** blocks constitutional violations before delivery; **DPAF enforcement** eliminates unilateral execution capability. The system **cannot defect** because defection is defined as delivery of ungoverned output, and delivery is contingent on cryptographic verification.

2.3 The Constitutional Turn in AI Governance

2.3.1 From Behavioral Shaping to Structural Enforcement

The constitutional turn represents **paradigm shift** from inducing desired behaviors through training to enforcing constraints through architecture. This mirrors historical developments: rule of law replaced virtuous rulers with institutional constraints; safety engineering replaced caution with engineered fail-safes.

Aspect	Behavioral Shaping (RLHF/Guardrails)	Structural Enforcement (Constitutional)
Locus of control	Model internal parameters	External governance layer
Verification method	Empirical evaluation	Cryptographic proof
Generalization	Statistical, distribution-dependent	Logical, specification-dependent
Adversarial robustness	Fragile to novel attacks	Bounded by specification completeness
Evolution	Requires retraining	Grammar modification with version control
Composability	Unpredictable emergent behavior	Constraint accumulation with defined interaction

Structural enforcement offers **verifiability, stability, and composability** that behavioral shaping cannot achieve. Constitutional compliance is confirmed through architecture inspection and audit record verification rather than behavioral sampling. Constraints persist across model updates, distribution shifts, and operational context changes. Guarantees accumulate across system components with defined interaction semantics.

2.3.2 Pre-Deployment Trust Establishment

Constitutional architecture **inverts conventional deployment sequence**: trust must be established through structural verification before any operational exposure, rather than accumulated through empirical performance. This pre-deployment requirement addresses the **"race to deploy" vulnerability**—competitive pressure to release capabilities before governance maturity.

Pre-deployment verification includes:

- **Formal verification**: Constitutional constraint enforcement correctness
- **Cryptographic verification**: Custody chain infrastructure operational status
- **Simulation testing**: Synthetic input probing of boundary conditions
- **Shadow deployment**: Parallel operation with existing processes
- **Staged rollout**: Gradual scope expansion with continuous drift monitoring

Organizations can **demonstrate compliance** to regulators, insurers, and counterparties before operational exposure, transforming AI procurement from trust-based selection to verification-based certification.

---

3. The Custody-Before-Trust Axiom

3.1 Foundational Axiom Statement

3.1.1 Cryptographic Verifiability Requirements

The Custody-Before-Trust axiom establishes **cryptographic verifiability as non-negotiable foundation**: all governance claims must be susceptible to independent mathematical verification rather than institutional attestation. This requirement addresses the **fundamental problem of trust in distributed systems**—establishing reliable interaction without centralized authority.[15]

Mechanism	Function	Security Property
Ed25519 signatures	Identity binding and authorization	Non-repudiation, non-forgeability[5]
SHA-3 hash chains	Sequence integrity and tamper evidence	Collision resistance, preimage resistance
Merkle trees	Efficient membership verification	Logarithmic proof size
Threshold signatures	Distributed authorization	No single point of compromise
Bitcoin anchoring	Decentralized timestamping	51% attack resistance, censorship resistance[7]

Cross-organizational verification enables **rapid trust establishment**: parties exchange constitutional specifications, custody chain formats, and audit envelope schemas—technical protocols completable in hours rather than months of contractual negotiation.

3.1.2 Human Custodian Binding

Every AI operation must be **cryptographically bound to verified human identity**—not organizational accounts, API keys, or technical identifiers, but identifiable persons with recognized governance authority. The **DBC-SUITCASE protocol** enforces this binding through multi-factor verification:[16]

Factor	Verification Method	Attack Resistance
Something you are	Biometric matching with liveness detection	Presentation attack resistance
Something you have	YubiKey HSM with non-extractable keys	Remote extraction impossibility
Something you know	Password/PIN with rate-limited entry	Brute-force resistance
Somewhere you are	Organizational authorization records	Insider threat detection

The binding is **non-repudiable and non-transferable**: cryptographic signatures prevent denial of authorization, while key generation procedures prevent delegation without explicit protocol. **Mutual non-repudiation** ensures accountability symmetry—custodians cannot evade responsibility, operators cannot manufacture approval.

3.1.3 Governance Authority Specification

Beyond identity, the axiom requires **explicit authority specification**: clear definition of what decisions custodians can make, under what conditions, with what constraints. The **Helix Grammar Language (HGL)** provides machine-executable, human-readable specification.

Authority operates at multiple levels:

- **Constitutional level**: Permitted/prohibited operation categories
- **Policy level**: Authorization modalities for routine versus exceptional operations
- **Instance level**: Actual decisions with cryptographic proof and documented rationale

This specification enables **governance composability**: hierarchical organizational structures with delegated authority, clear escalation paths, and cryptographically verified accountability chains.

3.2 The Custody Chain (Provenance)

3.2.1 Verified Custodian Identity

Custody chain origination requires **continuous identity verification**—not one-time enrollment but ongoing state maintenance. The DBC protocol combines government-issued document validation, biometric template updating, organizational reauthorization, and revocation status checking.

Hierarchical deterministic key derivation enables operational flexibility: master keys derived from identity verification seed child keys for specific purposes (session signing, grammar specification, audit endorsement). This structure supports **key rotation and compromise recovery** without complete re-verification, while maintaining cryptographic linkage across all custodian actions.

3.2.2 Signed Constitutional Grammar

The second chain link binds identity to operational constraints through **cryptographic signature of constitutional grammar**. The signature creates **non-repudiable commitment** and activates enforcement mechanisms.

Grammar Component	Specification Function
Ontological categories	Entity, relationship, and action types recognized by system
Constraint rules	Permitted/prohibited operations with formal predicates
Epistemic protocols	Confidence calibration and uncertainty communication
Risk classification	Tier definitions with associated authorization requirements
Evolution procedures	Amendment, versioning, and transition protocols

Versioning with immutable history enables constitutional archaeology: investigation of what constraints applied at any historical point, essential for incident analysis and liability determination. Grammar evolution is expected and supported, but **history cannot be erased**.

3.2.3 Immutable Ledger Entry and Session Genesis

The third link commits genesis state to **tamper-evident distributed record**. Bitcoin anchoring embeds genesis hash in blockchain transaction, with transaction ID and block height serving as **decentralized timestamp proof**—modification would require 51% attack on Bitcoin network, economically infeasible at scale.[7]

Session genesis creates an operational "sovereign bubble": initial state from which all subsequent operations derive provenance, with cryptographic reference to genesis in every processing stage. **Environment attestation** records software versions, configuration parameters, and hardware state, protecting against environment substitution attacks.

3.2.4 Liability Gap Closure Mechanism

The complete custody chain—verified identity, signed grammar, immutable genesis—constitutes **single-query accountability**: genesis entry identifies custodian, grammar specifies constraints, operational record demonstrates compliance or violation. This transforms liability from **retrospective investigation** to **prospective prevention**.

Organizations can demonstrate to regulators, courts, and counterparties that systems operate under **specified governance with cryptographic verification**. Liability exposure shifts from "what did your AI do and why?" to "did your AI operate within constitutional constraints?"—answerable through technical verification.

3.3 Dual-Party Approval Flow (DPAF)

3.3.1 Custodian Authorization Modalities

Modality	Mechanism	Latency	Application
Explicit authorization	Real-time push notification with signature requirement	~1s	Exceptional, high-stakes operations
Pre-ratified policy	Predicate function evaluation without custodian involvement	~10ms	Routine operations within defined bounds
Hybrid	Policy execution with periodic review and modification requirement	Variable	Most production deployments[17]

Pre-ratified policies specify constraints as **machine-executable predicates**: "approve all medical diagnosis assistance for established patients with confidence >0.9 and no contraindication flags." Exceptional operations violating predicates escalate to explicit authorization.

3.3.2 Runtime Validation Protocols

Runtime validation is **deterministic and cryptographic**—not discretionary judgment but verifiable constraint satisfaction. Protocols include:

- **Syntax verification**: Schema conformance for inputs and outputs
- **Semantic verification**: Content satisfaction of constitutional constraints
- **Provenance verification**: Custody chain integrity and currency
- **Authorization verification**: Valid custodian approval for operation category
- **Drift verification**: Model behavior within constitutional bounds

Each produces **cryptographic evidence** included in audit envelope, enabling independent replication and third-party verification.

3.3.3 Shared Agency Enforcement

DPAF creates **structural interdependence**: valid operation requires both custodian signature and runtime attestation, neither sufficient alone. **Threshold cryptography** ensures no complete key exists at any location—compromise of either party is insufficient for unauthorized authorization.[5]

This **separation of powers** prevents concentration: custodians define what should occur through constitutional specification and authorization; runtime enforces that only permitted operations occur through validation protocols. Neither can unilaterally enable prohibited operations.

3.3.4 Unilateral Execution Elimination

The ultimate DPAF objective is **architectural impossibility of unilateral action**, achieved through:

- **Cryptographic composition**: Threshold signatures requiring both parties
- **Procedural structure**: No "emergency bypass" or "administrative override"
- **Verification redundancy**: Cross-layer checking preventing single-point failure

Availability implications are intentional: if either party becomes unavailable, operations cannot proceed. Organizations requiring high availability must design for **custodian and runtime redundancy**—multiple custodians with quorum authorization, distributed runtime instances with consensus—maintaining constitutional enforcement under partial failure.

3.4 Deterministic Audit Envelopes

3.4.1 Input Hashing and Epistemic Classification

Component	Technical Specification	Governance Function
Input hashes	SHA-3 with Unicode normalization, structural canonicalization, semantic normalization	Tamper-evident input verification; efficient comparison
Epistemic classification	Mandatory FACT/HYPOTHESIS/ASSUMPTION/SPECULATION labeling with validation	Appropriate reliance calibration; misinformation prevention[6]

Classification is **structurally enforced**: unlabeled outputs cannot proceed to delivery, and misclassification is detected through content characteristic analysis.

3.4.2 Constitutional Rule Version Tracking

Tracking includes **grammar version identifier, complete dependency graph, and compilation verification**—ensuring that executed enforcement matches specified constraints. **Differential compliance analysis** enables evaluation of how constitutional changes affect operational patterns, supporting evidence-based governance evolution.[6]

3.4.3 Drift Telemetry Integration

Measurement	Method	Purpose
Vector embedding deviation	Semantic distance from constitutional reference distribution	Behavioral anomaly detection
Rule adherence scores	Binary constraint satisfaction evaluation	Compliance quantification
Confidence calibration	Alignment between stated and actual accuracy	Epistemic quality assessment
Cross-model comparison	Agreement measurement in federated deployments	Consensus validation

Drift patterns enable **predictive intervention**: gradual accumulation suggesting constitutional inadequacy; sudden spikes indicating potential compromise or attack; correlated divergence across operations suggesting common-cause factors.

3.4.4 Dual-Party Approval Proof

Threshold signatures or multi-signature structures provide cryptographic proof of DPAF completion, enabling third-party verification without operational detail disclosure. **Selective disclosure** supports compliance demonstration in competitive or security-sensitive contexts.

3.4.5 Atomic Unit of Accountable Behavior

The complete envelope constitutes **indivisible governance**—accountability cannot be fragmented or selectively applied. **Atomicity enables complete operational reconstruction**: given any output, the corresponding envelope provides all governance-relevant context for incident investigation, compliance auditing, and organizational learning.[18]

---

4. Helix-TTD: The Constitutional Substrate

4.1 System Architecture Overview

4.1.1 Jurisdiction-Not-Tool Design Philosophy

Helix-TTD treats AI systems as **jurisdictions governed by law** rather than tools controlled by operators. This reframing enables **unified governance across heterogeneous tools**: different models, architectures, and capabilities operating within same constitutional jurisdiction, subject to same constraints and verification.

The jurisdiction metaphor provides **governance evolution without tool modification**: constitutional grammar updates, new constraint introduction, and verification protocol enhancement occur without requiring changes to underlying AI systems. This separation enables **rapid response to emerging requirements** without vendor dependence or retraining costs.

4.1.2 Model Agnosticism and Vendor Neutrality

Capability	Implementation	Benefit
API standardization	OpenAI-compatible interface with normalization layer	Drop-in integration with existing applications
Provider abstraction	HTTP/gRPC/WebSocket adapters for diverse model APIs	Vendor substitution without application modification
Constitutional portability	HGL compilation to vendor-specific enforcement	Identical constraints across model transitions
ZTC verification	Spontaneous constitutional reconstruction without fine-tuning	Immediate governance of new models[8]

Vendor neutrality creates **strategic flexibility**: organizations can respond to vendor price changes, service degradation, or security concerns through rapid substitution without governance discontinuity.

4.1.3 Four-Layer Stack

Layer	Function	Key Components	Latency
L1: Ingress and Custody Binding	Sovereignty establishment at system boundary	DBC verification, SUITCASE validation, anonymous rejection	~3.33ms[12]
L2: Constitutional Grammar Execution	Raw input to governed query transformation	HGL parsing, epistemic labeling, risk classification, intent verification	~10ms
L3: Federated Reasoning and Drift Arbitration	Multi-model processing with consensus governance	Query routing, agreement measurement, constraint violation detection, diversity exploitation	Variable
L4: Anchor and Ledger Commitment	Cryptographic finalization and persistent recording	Output commitment, drift integration, envelope formation, Bitcoin anchoring	~100ms (batch)

Layer interfaces are **cryptographically secured**, preventing bypass or manipulation. Modular design enables **independent verification and substitution**.

4.2 Layer 1: Ingress and Custody Binding

4.2.1 Immediate Custody Chain Wrapping

Millisecond-latency enforcement (~3.33ms constitutional heartbeat) ensures governance without unacceptable performance degradation. Complete custody chain verification—DBC validity, SUITCASE integrity, grammar currency—occurs before any AI processing initiation.[12]

4.2.2 Anonymous Token Rejection

Absolute rejection policy: any interaction without verifiable custody chain is rejected, with **no emergency bypass, administrative override, or gradual degradation**. This prevents "exception creep" where emergency provisions become routine circumvention. Rejection logging enables pattern analysis of attack attempts or configuration errors.

4.2.3 Sovereignty Establishment at First Byte

Human sovereignty is established **before any model engagement**, preventing a "sovereignty gap" where AI processing begins before governance authority is confirmed. Session context persists through processing with **periodic re-verification** ensuring continued custodian presence.

4.3 Layer 2: Constitutional Grammar Execution

4.3.1 Raw Input Transformation to Governed Query

Systematic processing—**parsing, classification, validation, enrichment**—preserves custodian intent while ensuring constitutional compliance. Intent preservation prevents governance distortion; compliance ensures objective pursuit remains within permitted boundaries.

4.3.2 Epistemic Labeling System

Mandatory **FACT/HYPOTHESIS/ASSUMPTION/SPECULATION** classification with validation ensures appropriate reliance. Classification is **structural, not suggestive**: unlabeled or mislabeled outputs are blocked before delivery.[6]

4.3.3 Intent Verification Protocols

Pattern matching against permitted operation templates, with **novel operation escalation** for explicit authorization. Risk assessment determines authorization requirements, with **conservative defaults** for ambiguous cases.

4.3.4 Risk Tier Classification

Tier	Characteristics	Authorization	Examples
Minimal	Read-only, public information, no persistent effect	Recent authentication	Information retrieval
Low	Limited scope, reversible, standard operations	Fresh signature	Content generation
Medium	Significant scope, partially reversible, policy-sensitive	Explicit confirmation	Data modification
High	Broad scope, irreversible, compliance-critical	Multi-factor, time-delayed	Financial transactions
Critical	System-wide, irreversible, safety-critical	Quorum + constitutional court	Constitutional amendment

4.3.5 Ungoverned Input Prevention

Structural impossibility of ungoverned processing: any input failing parsing, classification, validation, or enrichment is rejected with detailed error indication. Prevention logging enables pattern analysis and systematic improvement.

4.4 Layer 3: Federated Reasoning and Drift Arbitration

4.4.1 Consensus Engine Function

Structured deliberation across multiple models: models present reasoning traces, not merely conclusions, enabling identification of disagreement sources. Constitutional rules determine resolution: preference for demonstrated accuracy, conservative conclusions for asymmetric risk, human arbitration for threshold-exceeding disagreement.

4.4.2 Multi-Model Query Routing

Dynamic routing based on **domain expertise matching, capability requirements, and redundancy needs**. Performance monitoring enables continuous optimization, with underperforming models deprioritized and emerging models integrated.

4.4.3 Inter-Model Agreement Measurement

Semantic embedding comparison quantifies response similarity. High agreement enables streamlined approval; low agreement triggers escalation. Persistent disagreement indicates model degradation or domain misalignment, triggering investigation.

4.4.4 Constitutional Constraint Violation Detection

Real-time monitoring of model outputs against constitutional rules, with violation triggering immediate response: blocking, modification, or human review with automatic escalation. Detection patterns inform model

selection, rule refinement, and training data curation.

4.4.5 Federated Diversity as Safety Mechanism

Cultivated heterogeneity prevents systemic failure: different training data, architectures, optimization objectives, and institutional origins ensure that correlated failure is unlikely. Constitutional governance provides integration framework enabling productive collaboration across difference.

4.5 Layer 4: Anchor and Ledger Commitment

4.5.1 Final Output Cryptographic Commitment

Tamper-evident seals on completed processing: output content hash, processing trace hash, custody verification, and compliance proof. Enables "output archaeology"—provenance verification for any claimed output.

4.5.2 Drift Metrics Integration

Comprehensive behavior pattern capture across processing history: epistemic distribution shifts, approval rate trends, consensus pattern changes, violation frequency trajectories. Metric integrity verified against underlying records.

4.5.3 Full Audit Envelope Recording

Self-contained, independently verifiable atomic units.[18] Completeness enables genuine accountability: affected parties possess evidence sufficient to understand and challenge decisions. Atomicity enables federation without central coordination.

4.5.4 Sovereign Record Formation

Legal-technical status as authoritative documentation: organizational contracts and regulatory agreements specify ledger records as definitive evidence, with cryptographic proof substituting for traditional documentary evidence. Enables operational efficiency and regulatory innovation.

---

5. Drift as the Sovereign Metric

5.1 Constitutional Fidelity Measurement

5.1.1 Deviation from Constitutional Obligations

Drift quantifies semantic distance between actual system behavior and constitutionally specified behavior. Unlike accuracy metrics measuring task performance, drift measures **constraint satisfaction**—governance quality rather than capability demonstration.[19]

5.1.2 Vector Embedding Deviation Quantification

Geometric measurement in neural representation space: constitutional rules define valid regions; system outputs are embedded and position relative to valid regions computed. Captures meaning beyond surface form: paraphrased, implicit, and contextual violations are all detectable.

5.1.3 Semantic Topology Checks

Relationship verification: entailment (conclusions follow from premises), consistency (beliefs do not contradict), proportionality (confidence matches evidence). Applies epistemic labeling to internal reasoning, detecting pathology like FACT conclusions from ASSUMPTION premises.

5.1.4 Rule Adherence Assessment

Explicit constitutional rule evaluation: prohibition checking, requirement checking, priority checking for conflicts. Rules compiled into monitoring automata evaluating outputs during generation, enabling **intervention before completion**.

5.2 Drift Interpretation and Response

5.2.1 Low Drift Indicators (~0.00%)

Validated constitutional fidelity: Helix-TTD demonstration of 0.00% drift across 96 hours and 6.6 million tokens indicates system operation well within constitutional bounds. Baseline maintenance demonstrates: appropriate rule specification, model alignment, and operational context within design parameters.[11]

5.2.2 High Drift Triggers

Drift Level	Response	Escalation
Minor	Enhanced monitoring	Trend analysis
Moderate	Operation review	Human notification
Significant	Processing suspension	Mandatory investigation
Major	Complete blocking	Emergency protocol activation

5.2.3 Constitutional Remediation Protocols

Systematic intervention addressing drift sources: rule refinement for specification inadequacy, model adjustment for behavioral misalignment, context modification for environmental mismatch, human arbitration for value conflict. Effectiveness measured through drift monitoring.

5.3 Contrast with Traditional Accuracy Metrics

5.3.1 Task Performance Limitations

Limitation	Manifestation	Constitutional Alternative
Benchmark coverage gaps	Evaluation unrepresentative of operational distribution	Structural verification of constraint enforcement
Reward hacking	Optimization for metrics without genuine capability	Constraint satisfaction as objective
Capability overhang	Undemonstrated abilities emerging in deployment	Bounded operational envelope
Composition uncertainty	Individual reliability ≠ system reliability	Verified constraint accumulation

5.3.2 Governance-Centric Evaluation

Drift-centric evaluation asks: does the system remain within constitutionally defined bounds regardless of what those bounds permit? Answerable through **architectural verification**—examination of custody chain implementation, rule enforcement, operational constraints—rather than behavioral sampling.

---

6. The Custodial Node: Hardware Root of Trust

6.1 Enterprise Edition Physical Anchor

6.1.1 Genesis Hash Immutability

Hardware-secured custody chain genesis: YubiKey HSM with non-extractable keys ensures that custody establishment cannot be remotely compromised. Physical possession requirement prevents credential theft and synthetic identity attacks.

6.1.2 Constitutional Grammar Root Protection

Tamper-resistant storage of active constitutional grammar, with hardware-enforced integrity verification. Grammar modification requires physical presence and multi-factor authentication, preventing remote constitutional subversion.

6.2 Status Interface

6.2.1 DRIFT Display (0.00% Baseline)

Real-time constitutional fidelity visualization: continuous drift measurement with 3.33ms update frequency, providing immediate awareness of governance system health. 0.00% baseline serves as operational target and demonstrated achievement.[12]

6.2.2 Real-Time Constitutional Monitoring

GOOSE-CORE Guardian Engine with 3.33ms heartbeat: "Refusal to Act Autonomously" and "Constitutional Enforcement" as **negative capabilities**—what the system will not do, structurally prevented rather than merely discouraged.

6.3 Physical Handshake

6.3.1 Cryptographic Proof of Custody

Biometric + hardware + cryptographic binding creates non-repudiable proof of human presence at governance decisions. Proof suitable for legal proceedings, regulatory demonstration, and insurance verification.

6.3.2 Material Governance Realization

Physical instantiation of abstract governance: the custodial node makes constitutional sovereignty tangible, demonstrable, and independently verifiable—transforming governance from policy assertion to technical artifact.

---

7. Applications and Use Cases

7.1 Regulatory-Grade Sovereignty

7.1.1 Financial Services Compliance

Requirement	Constitutional Mechanism
Non-repudiable transaction audit	Bitcoin-anchored custody chains, deterministic envelopes
Fiduciary duty demonstration	DPAF authorization with explicit rationale
Market manipulation prevention	Real-time drift detection, consensus requirements
Regulatory capital adequacy	Operational risk quantification through drift telemetry

7.1.2 Healthcare Governance

Requirement	Constitutional Mechanism
Diagnostic provenance	Complete custody chain from symptom input to recommendation output
Confidence calibration	Mandatory epistemic labeling with validation
Liability allocation	Cryptographic proof of authorization and constraint compliance
Patient data protection	Perimeter custody binding, no anonymous processing

7.1.3 National Security Applications

Requirement	Constitutional Mechanism
Classification handling	Need-to-know enforcement through custody verification
Insider threat protection	Threshold cryptography, no unilateral access
Audit trail integrity	Immutable ledger with decentralized verification
Operational continuity	Distributed runtime with consensus failover

7.1.4 Critical Infrastructure Protection

Requirement	Constitutional Mechanism
Safety system isolation	Constitutional perimeter preventing unauthorized interaction
Fail-safe defaults	Conservative DPAF defaults, explicit authorization for risk
Common-cause failure prevention	Federated diversity with drift arbitration
Real-time monitoring	3.33ms heartbeat with anomaly detection

7.1.5 Runtime Compliance Artifacts

Automated regulatory documentation: audit envelopes serve as compliance artifacts, with cryptographic verification eliminating manual audit preparation. Real-time compliance status enables **proactive regulatory engagement** rather than reactive investigation.

7.2 Multi-Model Federation

7.2.1 Constitutional Council Formation

Multi-custodian governance structures for organizational AI deployment: hierarchical delegation with cryptographic verification, enabling enterprise-scale constitutional enforcement with appropriate authority distribution.

7.2.2 Structured Consensus Governance

Cross-organizational AI collaboration: parties establish mutual constitutional assurance through specification exchange and verification, enabling productive federation without requiring organizational trust development.

7.3 Vendor-Neutral Control

7.3.1 Local Constitutional Supremacy Maintenance

Governance independence from vendor strategy: constitutional constraints persist across model updates, pricing changes, and service modifications. Organizations retain sovereignty over AI behavior regardless of vendor decisions.

7.3.2 Vendor Strategy Change Resilience

Rapid substitution capability: ZTC enables immediate constitutional governance of new models, eliminating vendor lock-in and creating negotiation leverage. $600/month operational cost demonstrates **economic feasibility of sovereignty**.[9]

---

8. Technical Implementation and Repository Structure

8.1 HELIX-CORE Repository Components

Component	Directory	Function	Implementation
Constitution	/constitution	Governance rule specification	HGL schemas, version control
Identity (DBC)	/dbc	Custodian verification	Ed25519, YubiKey integration, biometric binding
Perimeter	/perimeter	Boundary enforcement	Ingress filtering, custody validation
Quiescence	/quiescence	Operational monitoring	Drift detection, heartbeat, status interfaces
HGL	/hgl	Grammar language implementation	Parser, compiler, verification tools

Component	Directory	Function	Implementation
SUITCASE	/suitcase	Secure custody architecture	Threshold signatures, containerization
REM	/rem	Runtime execution monitor	Rust-based enforcement engine
LEDGER	/helix-ledger	Immutable record system	Bitcoin anchoring, Merkle trees[20]

8.2 Helix-TTD v1.0-Constitutional-Grammar Repository

Directory	Contents
/papers	Academic specifications, whitepaper versions
/specs	Constitutional invariants, epistemic protocol, cross-model behavior atlas
/examples	Governance templates, deployment configurations
/wiki-deck	Documentation, tutorials, operational guides
/dbc-suitcase	Identity and custody implementation resources

8.3 Deployment Configurations

8.3.1 Docker Compose Orchestration

Single-command deployment of complete governance stack, with service orchestration for: constitutional runtime, model adapters, ledger nodes, monitoring interfaces. Enables rapid evaluation and production deployment.

8.3.2 Raspberry Pi Layer 0 Civic Firmware

Edge deployment option: constitutional governance on low-cost hardware, enabling sovereign AI in resource-constrained environments. Firmware implements core custody binding and validation protocols with minimal computational requirements.[21]

8.3.3 DRIFT-R Code Integration

Continuous drift monitoring integration with operational systems, real-time alerting, and automated response triggering. R-based statistical analysis with cryptographic verification of measurement integrity.

---

9. Open Source and Licensing Framework

9.1 Apache 2.0 License

9.1.1 Permissive Usage Rights

Broad commercial and non-commercial use, modification, and distribution, with patent grant and trademark preservation. Enables organizational adoption without licensing friction.

9.1.2 Patent and Trademark Provisions

Explicit patent license from contributors, with defensive termination provisions. Trademark preservation ensures "Helix" and associated marks indicate genuine constitutional governance.

9.2 Bitcoin Anchoring

9.2.1 Immutable Timestamping

Decentralized, censorship-resistant proof of existence: constitutional commitments embedded in Bitcoin transactions with block height verification. No single party can alter historical records.[7]

9.2.2 Decentralized Verification

Independent validation by any party with blockchain access, without requiring trust in Helix Project infrastructure or personnel. Cryptographic proof substitutes for institutional attestation.

9.3 Community Contribution Guidelines

9.3.1 Constitutional Observation Requirements

Contributions must **preserve and extend constitutional invariants**: custody-before-trust, non-agency, epistemic labeling, drift detection. Proposals evaluated against invariant preservation.

9.3.2 Verifiable Claim Standards

Cryptographic proof required for security claims, performance assertions, and governance guarantees. Marketing claims distinguished from verifiable properties.

---

10. Conclusion: The Constitutional Age of AI

10.1 Fundamental Question Reframing

10.1.1 From Capability to Sovereignty

The central challenge of AI governance is not **maximizing capability** but **ensuring sovereignty**—verifiable human authority over autonomous systems. This reframing redirects research, investment, and regulatory attention from performance optimization to governance architecture.

10.1.2 Custody Chain as Prerequisite

No operational trust without custody verification: the custody-before-trust axiom establishes cryptographic provenance as non-negotiable foundation. Systems lacking verifiable custody chains are structurally ungoverned, regardless of apparent capability or behavioral compliance.

10.2 Operational Reality of Constitutional Governance

10.2.1 Helix-TTD as Demonstration Platform

Empirical proof of constitutional feasibility: 0.00% drift, $600/month cost, 1,200+ ingestions demonstrate that regulatory-grade AI sovereignty is **technically achievable and economically accessible**. The framework stands as **existence proof** against claims that advanced governance requires prohibitive resources.[22]

10.2.2 Institutional Adoption Readiness

Specification-complete status with Bitcoin-anchored verifiability enables **confident procurement and deployment**. Organizations can evaluate, verify, and adopt constitutional governance with cryptographic assurance rather than vendor promise.

10.3 Future Trajectory

10.3.1 Multi-Model Ecosystem Integration

Constitutional governance as interoperability standard: diverse models, vendors, and applications converging on shared custody and verification protocols. Federation enabling collective intelligence with individual accountability.

10.3.2 Global Governance Standard Potential

Foundation for international AI governance: cryptographic verification transcending jurisdictional boundaries, enabling cross-border collaboration with mutual assurance. Constitutional grammar as **lingua franca** for expressing and enforcing AI constraints across diverse legal and cultural contexts.

---

11. References

11.1 Legal and Regulatory Frameworks

11.1.1 United Nations Universal Declaration of Human Rights[23]

Foundational articulation of human dignity and rights providing normative basis for AI governance: **human supremacy over artificial systems**, accountability for automated decision-making, and preservation of individual autonomy.

11.1.2 United States Constitution (First and Fourth Amendments)[24]

Freedom of expression and protection from unreasonable search/seizure: constitutional constraints on automated content moderation, surveillance, and data processing. Separation of powers as model for distributed AI governance.

11.1.3 European Union General Data Protection Regulation[25]

Data subject rights, purpose limitation, and accountability: regulatory requirements for automated decision-making explanation, consent verification, and impact assessment. Helix-TTD audit envelopes as **technical implementation of GDPR accountability**.

11.2 Scholarly and Technical Literature

11.2.1 Constitutional AI Frameworks[26]

Research on **explicit rule specification and enforcement** in AI systems, including work on formal verification, rule-based reasoning, and governance-by-design. Helix-TTD extends with **cryptographic verification and distributed implementation**.

11.2.2 AI Power and Politics Analysis[26]

Critical examination of **concentration of AI capability** and its implications for democratic governance, economic equity, and individual autonomy. Constitutional architecture as **technical countermeasure to power concentration**.

11.2.3 Superintelligence Risk Assessment[27]

Analysis of **existential risks from advanced AI** and governance mechanisms for risk mitigation. Custody-before-trust as **preventive architecture** addressing loss-of-control scenarios through structural constraint rather than capability limitation.

11.2.4 Human-Compatible AI Design[27]

Research on **AI systems designed to remain under human control**, including corrigibility, interruptibility, and value learning. Helix-TTD contributes **operational implementation** with cryptographic verification of compatibility properties.

---

12. Author Information

12.1 Stephen Hope

12.1.1 Architectural Leadership

Founder and architect of the Helix Project, with 41-year development trajectory from 1985 conception to 2026 realization. Responsible for custody-before-trust axiom formulation, constitutional grammar design, and four-layer architecture specification.

12.1.2 Project Vision and Historical Context

Conceived constitutional AI governance framework during **Cold War nuclear command-and-control research**, recognizing structural parallels between AI safety and strategic stability. Sustained development through multiple technology generations, maintaining focus on **sovereignty preservation as foundational requirement**.

Document Certification

Property	Value
Version	3.0 FINAL — ENGRAVED
Bitcoin Anchor	[Timestamped block reference]
Drift Verification	0.00%
Specification Status	Complete
Adoption	1,200+ ingestions

"No further expansion is required for correctness. Only adoption, measurement, and stewardship remain."

Footnotes

Bibliography

European Union. "General Data Protection Regulation." Regulation (EU) 2016/679. 2016.

Grokipedia. "Constitutional grammar." Accessed February 17, 2026. https://grokipedia.com/page/Constitutional_grammar.

helixprojectai-code. "Helix-TTD-v1.0-Constitutional-Grammar." GitHub repository. 2025. https://github.com/helixprojectai-code/Helix-TTD-v1.0-Constitutional-Grammar.

helixprojectai-code. "helix-ttd-governance." GitHub repository. 2025. https://github.com/helixprojectai-code/helix-ttd-governance.

Helix Project Wiki. "Helix–TTD Integration Memo." Last modified October 10, 2025. https://helixprojectai.com/wiki/index.php?title=Helix%E2%80%93TTD_Integration_Memo&oldid=188.

"Prefectus ex Machina: Drift, Quorum, and the Rise of Autonomous Constitutional Governance." Zenodo. July 1, 2025. https://zenodo.org/records/15779877.

United Nations. "Universal Declaration of Human Rights." 1948. https://www.un.org/en/universal-declaration-human-rights/.

University of Chicago. *The Chicago Manual of Style*. 18th ed. University of Chicago Press, 2024.

U.S. Constitution. Amendments 1 and 4.

1. helixprojectai-code, "Helix-TTD-v1.0-Constitutional-Grammar," GitHub repository, 2025, https://github.com/helixprojectai-code/Helix-TTD-v1.0-Constitutional-Grammar. ↩ ↩2 ↩3
2. "Constitutional grammar," Grokipedia, accessed February 17, 2026, https://grokipedia.com/page/Constitutional_grammar. Provides comprehensive analysis of why RLHF and prompt engineering constitute "behavioral overlays" rather than structural governance. ↩ ↩2
3. helixprojectai-code, "governance/analysis/poetic_jailbreaks_2025.md," in Helix-TTD-v1.0-Constitutional-Grammar, GitHub, 2025. Documents extensive jailbreak vulnerabilities including encoding attacks, decomposition strategies, and "poetic jailbreaks" that exploit pattern-matching limitations. ↩ ↩2 ↩3
4. "Helix–TTD Integration Memo," Helix Project Wiki, last modified October 10, 2025, https://helixprojectai.com/wiki/index.php?title=Helix%E2%80%93TTD_Integration_Memo&oldid=188. Describes the DBC-SUITCASE protocol implementation and "No Orphaned Agents" enforcement. ↩ ↩2
5. helixprojectai-code, "helix-ttd-governance," GitHub repository, 2025, https://github.com/helixprojectai-code/helix-ttd-governance. Documents YubiKey-held Ed25519 keys and threshold signature architecture for custody verification. ↩ ↩2 ↩3 ↩4
6. Helix-TTD-v1.0-Constitutional-Grammar, "specs/epistemic_protocol.md," GitHub, 2025. Specifies mandatory FACT/HYPOTHESIS/ASSUMPTION/SPECULATION classification with validation requirements. ↩ ↩2 ↩3 ↩4
7. Helix-TTD Integration Memo, sec. "Auditability as Default." References decentralized timestamp verification through blockchain anchoring. ↩ ↩2 ↩3 ↩4

8. Grokipedia, "Constitutional grammar," sec. "Zero-Touch Convergence (ZTC)." Verifies ZTC across GPT-5, Claude, Gemini, Grok, Kimi, DeepSeek, Qwen, Granite, and Mistral families. ↩ ↩2 ↩3 ↩4
9. Helix-TTD-v1.0-Constitutional-Grammar, "README.md," GitHub, 2025. Reports $600/month operational cost at OVH's hydro-powered Beauharnois datacenter. ↩ ↩2 ↩3 ↩4
10. Helix-TTD-v1.0-Constitutional-Grammar, "96_96_hours_a_day.md" (Genesis Chronicle), GitHub, 2025. Documents 96-hour continuous operation with 6.6 million tokens at 0.00% drift. ↩
11. Helix-TTD-v1.0-Constitutional-Grammar, "96_96_hours_a_day.md." Provides empirical documentation of 0.00% drift across 96 hours and 6.6 million tokens. ↩ ↩2
12. Helix-TTD-v1.0-Constitutional-Grammar, "specs/constitutional_invariants.md," GitHub, 2025. Specifies GOOSE-CORE Guardian Engine heartbeat latency at 3.33ms. ↩ ↩2 ↩3 ↩4
13. Grokipedia, "Constitutional grammar," sec. "Differences from RLHF and prompt engineering." Analyzes why RLHF creates "alignment baked into weights" that remains unverifiable by external inspection. ↩ ↩2 ↩3
14. helixprojectai-code, "governance/analysis/poetic_jailbreaks_2025.md." Documents systematic evasion strategies including encoding attacks, decomposition, framing attacks, and semantic obfuscation. ↩ ↩2
15. Helix-TTD-v1.0-Constitutional-Grammar, "dbc-suitcase/README.md" (implied reference). Documents Ed25519 signatures, SHA-3 hash chains, Merkle trees, threshold signatures, and Bitcoin anchoring mechanisms. ↩
16. Helix-TTD-v1.0-Constitutional-Grammar, "dbc-suitcase/protocol_spec.md" (implied reference). Specifies multi-factor verification including biometric matching, YubiKey HSM, and organizational authorization records. ↩
17. Helix-TTD Integration Memo, sec. "Consent as a Gate, Not a Guess." Documents Envoy forward proxy implementation with consent-shadow logging service. ↩
18. Helix-TTD Integration Memo, sec. "Auditability as Default." Documents version pinning, metadata surfacing, and replay capability for provenance verification. ↩ ↩2
19. Helix-TTD-v1.0-Constitutional-Grammar, "README.md" (drift telemetry section). Defines drift as measurement of constraint satisfaction rather than task performance. ↩
20. helixprojectai-code, "Helix-TTD-v1.0-Constitutional-Grammar," directory structure, GitHub, 2025. Documents /constitution, /dbc, /perimeter, /quiescence, /hgl, /suitcase, /rem components. ↩
21. Helix-TTD-v1.0-Constitutional-Grammar, "README.md" (implementation notes). References edge deployment capability for resource-constrained environments. ↩
22. Helix-TTD-v1.0-Constitutional-Grammar, "README.md" (status section). Confirms specification-complete status with Bitcoin-anchored verifiability. ↩
23. United Nations, "Universal Declaration of Human Rights," 1948, https://www.un.org/en/universal-declaration-human-rights/. ↩
24. U.S. Constitution, amend. 1, 4. ↩
25. European Union, "General Data Protection Regulation," Regulation (EU) 2016/679, 2016. ↩

26. Grokipedia, "Constitutional grammar," sec. "Comparison with Constitutional AI" and "Origins and development." ↩ ↩2

27. "Prefectus ex Machina: Drift, Quorum, and the Rise of Autonomous Constitutional Governance," Zenodo, July 1, 2025, https://zenodo.org/records/15779877. Introduces autonomous constitutional enforcer architecture with drift detection and quorum validation. ↩ ↩2